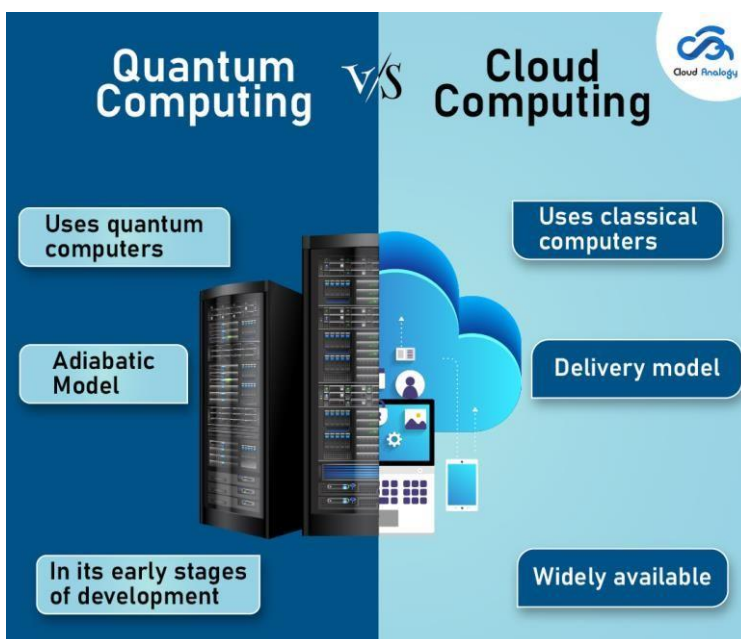




COMPUTACION PDN

PROYECTOS

La COMPUTACION Paralela, Distribuida y en la Nube – PDN; son conceptos que permiten maximizar el uso de los recursos de procesamiento (CPU, GPU, NPU); para optimizar los tiempos de respuesta de los algoritmos y el uso de grandes volúmenes de datos, orientados a Inteligencia Artificial, Simulación, Data Analítica y Sistemas en Tiempo Real. Incluye las tendencias: a QUANTUM y BIOLOGIC COMPUTING.

JW Cóndor

2026 marzo 24

Contenido

P1.1: Procesamiento Paralelo CPU Multinúcleo	7
Requerimientos Específicos:	7
Recomendaciones Técnicas	7
Rúbrica de Evaluación (50 puntos en total)	7
P1.2: Computación GPU Paralela	8
Requerimientos Técnicos Entrada:	8
Funcionalidades:	8
Criterios de Comparación	8
P1.3 computación CPU & GPU Paralela	10
P1.4 Computación NPU Paralela	11
II.- DISTRIBUTED COMPUTING	12
P2.1 Clúster de Computadoras	13
P2.2 Procesamiento Distribuido	15
P2.3 Procesamiento Distribuido Optimizado	17
Proyecto Integrador: "Ecosistema Genómico Heterogéneo y Distribuido"	17
1. Arquitectura del Sistema	17
2. Cronograma de Integración Semanal	17
Semana 1: El Motor de Control (CPU & Red)	17
Semana 2: Aceleración de Fuerza Bruta (GPU)	17
Semana 3: Inteligencia y Optimización (NPU)	17
4. Rúbrica de Evaluación Integral	17
5. Preparación del Entorno (Checklist):	18
6. Guía de Comandos UNIX	18
EXAMEN Distributed Computing:	18
2. Desarrollo y funcionalidad del sistema	37
3. Procesamiento de cadenas ADN 4GB	37

OBJETIVOS DE LOS PROYECTOS de Computación PDN:

1. **Motivar en el participante de computación PDN:**
 - a. La Investigación,
 - b. El Autoaprendizaje,
 - c. La Implementación y
 - d. El Trabajo colaborativo.
2. **Detallar las especificaciones de infraestructura tecnológica:**
 - a. Comunicaciones – ancho de banda
 - b. Hardware – almacenamiento capacidad.
 - c. Hardware – Procesamiento capacidad.
 - d. Firmware especializado.
 - e. Sistema Operativo – gestión de almacenamiento, procesamiento, memoria.
 - f. Lenguajes de Programación.
3. **Profundizar el conocimiento de:**
 - a. Computación Paralela (PARALLEL),
 - b. Computación Distribuida (DISTRIBUTED),
 - c. Computación en la nube (CLOUD),
 - d. Tendencias en cuanto a Computación (FOG, EDGE, QUANTUM, BIOLOGIC).

TIPOS DE EVIDENCIAS:

Una evidencia es un archivo digital tipificado como:

1. Informe,
2. Presentación,
3. Videoclip,
4. Código fuente, Script SQL, Sketch
5. Implementación.

1. Formato guía para evidencia tipo INFORME:

1. Documento en procesador de palabras
2. Tipo de letra Arial, 10 cpi. Margen normal de páginas.
3. Encabezado con el título del documento.
4. Pie de página con autor(es), numeración y fecha de exposición.
5. Estructura de informe:
 - a. Carátula: Tema, autor(es), Fecha de exposición.
 - b. Resumen
 - c. Desarrollo del tema.
 - d. Conclusiones y recomendaciones
 - e. Referencias y Bibliografía.

2. TIPS para PRESENTACION EFECTIVA:

1. Documento en presentador de diapositivas
2. Tamaño de letras visibles para el asistente de la última fila.
3. Cada diapositiva con título claro.
4. Pie de diapositiva con numeración y fecha.
5. Estructura de Presentación:
 - a. Caratula: Tema, autor(es), Fecha de exposición.
 - b. Desarrollo del Tema.
 - c. Conclusiones y Recomendaciones
 - d. Agradecimiento, preguntas y respuestas.
 - e. Referencias y Bibliografía.

3. TIPS para EXPOSICION de IMPACTO:

1. Escenario – conocer previamente el escenario donde se dará la exposición.
Por ejemplo: conocer la sala de grados, tamaño, capacidad, equipos tecnológicos.
2. Asistentes – conocer los asistentes que estará presente en la exposición.
Por ejemplo, en la defensa del Trabajo de Titulación, como mínimo el tribunal que son 3 docentes profesionales del área del conocimiento. Posiblemente las autoridades de la Carrera, facultad y/o Universidad, Gerentes o Empresarios, compañeros de aulas o de otras carreras, familiares y amigos.

3. Presentación – referirse al numeral #2 PRESENTACION EFECTIVA
4. Equipos – verificar el funcionamiento de los equipos: computador, proyector, control remoto, luces, conectividad.
5. Expositor:
 - a. Vestimenta – apropiada al evento.
 - b. Tono de voz – variar de acuerdo al auditorio, de tal manera que la última persona pueda escuchar. Enfatizar los conceptos, conclusiones o recomendaciones que se quiera hacer énfasis
 - c. Lenguaje corporal – movimiento de manos acorde a la exposición, énfasis con las indicaciones de las manos o con el puntero laser.
 - d. Lenguaje corporal – posición de cuerpo frente al auditorio, presencia serena y firme en los conceptos y conclusiones.
 - e. Vista – paneo del auditorio para ver reacciones de los mismos.
 - f. Control de nervios – técnica de clavar la uña en el dedo posterior hasta tomar confianza en la exposición.

4. Formato guía para VIDEO de proyecto con rúbrica de evaluación:

1. LINK al video en la plataforma EVA.
2. CADA integrante indica su apellido y presenta el criterio de evaluación.
3. A continuación, un guion del video clip:

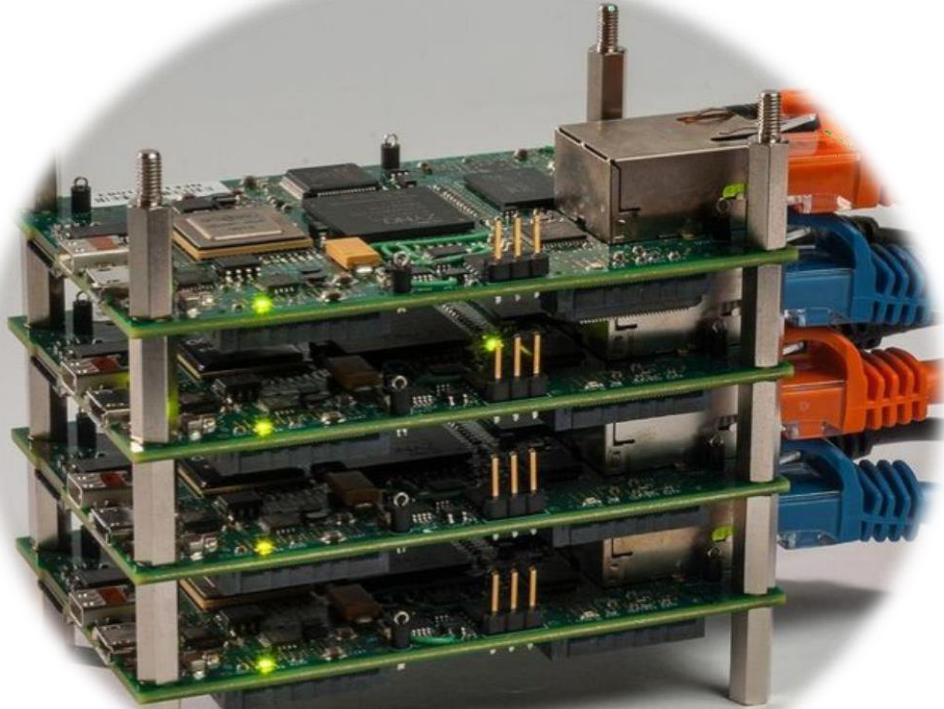
Seq:	Descripción:	Duración [mm:ss]
0.	Saludos [LIDER]	0:30
1.	Exposición criterio 1 [estudiante 1]	1:00 - 2:00
2.	Exposición criterio 2 [estudiante 2]	1:00 - 2:00
3.	Exposición criterio 3 [estudiante 3]	1:00 - 2:00
4.	Exposición criterio 4 [estudiante 4]	1:00 - 2:00
...	Mas criterios ...	1:00 - 2:00
6.	Conclusiones y Recomendaciones [LIDER]	1:00 - 2:00

5. Formato guía para IMPLEMENTACION:

1. Como infraestructura se puede utilizar maquinas físicas, máquinas virtuales, servidores locales, servidores en la nube.
2. Especificándose sus características y ubicación.
3. La ejecución del concepto debe ser aclarado para los asistentes.
4. Simularse condiciones similares al contexto geográfico, organizacional, transaccional.
5. Cualquier miembro del equipo se seleccionará para las modificaciones solicitadas.

CONDICIONES DE ENTREGA DE EVIDENCIAS:

1. La evidencia (archivo PDF) debe estar en la plataforma EVA.
2. La fecha y hora de la recepción está indicadas,
3. Cada día de retraso se penaliza con el 10%,
4. Cada falta ortográfica se penaliza con el 1%.
5. Desde enero del 2024:
 - a. Las soluciones, respuestas y/o propuestas del estudiante, deberán ser contrastadas con una inteligencia artificial [ej. chatGPT, CoPilot, Bard].
 - b. El estudiante determinará las variaciones existentes entre las propuestas por la inteligencia artificial y las propuestas por el estudiante.
 - c. El estudiante analizará el porqué de las variaciones.
6. La defensa del proyecto será calificada de acuerdo con la argumentación de las variaciones detectadas.



COMPUTACION PARALELA

I.- PARALELL COMPUTING

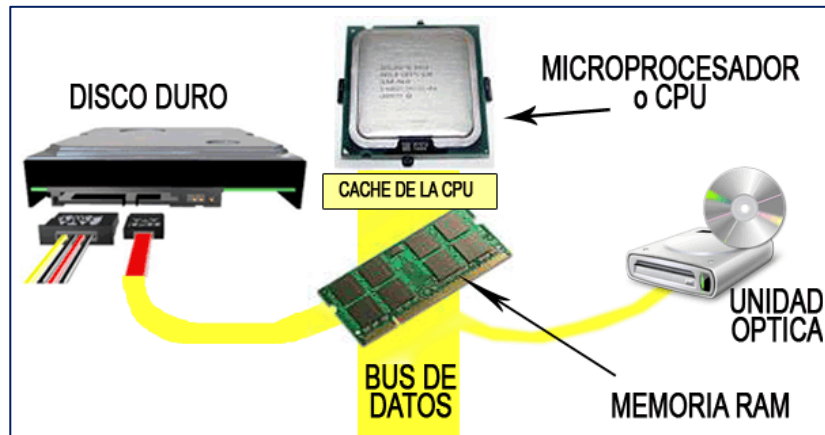
P 1.1 Computación CPU Paralela

PROCESAMIENTO SECUENCIAL

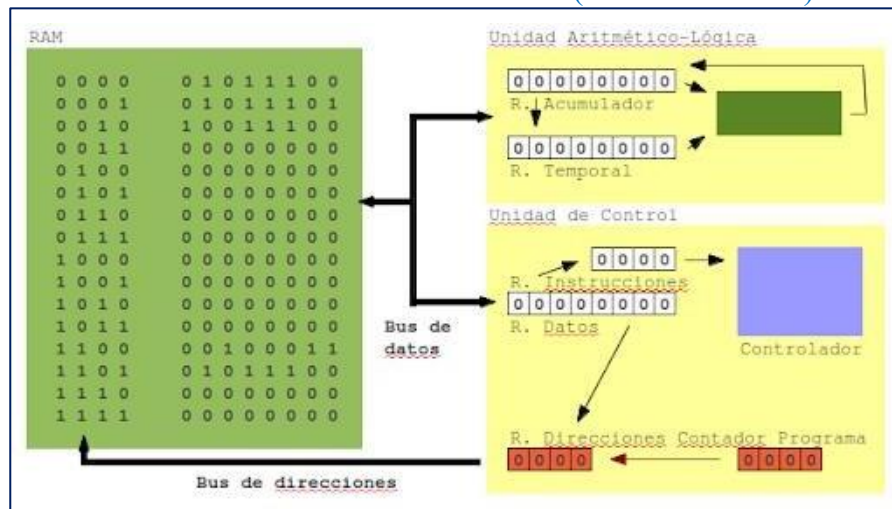
Se produce un procesamiento secuencial cuando una tarea va después de otra. Es un proceso lento en el que, si una tarea se retrasa, el sistema completo debe esperar. La ventaja es que es fácil de entender y de implementar.



COMPONENTES DE UN COMPUTADOR CON UN CPU:



EJECUCION SECUENCIAL DE UN PROGRAMA (CPU $\leftrightarrow$ RAM):



P1.1: Procesamiento Paralelo CPU Multinúcleo

Desarrollar una aplicación que procese un archivo de 4GB de cadenas de ADN, aplicando técnicas de computación paralela sobre CPU multinúcleo, y que permita comparar el desempeño entre ejecución secuencial y paralela, utilizando control explícito sobre los núcleos de la CPU.

Requerimientos Específicos:

1. **Desarrollar un programa** en C/C++ (recomendado por eficiencia) o Python, que procese una cadena de ADN (simulada o real, de 4GB), por ejemplo, contando la frecuencia de cada nucleótido (A, C, G, T).
2. Implementar dos versiones del procesamiento:
 - a. Versión secuencial (1 solo núcleo).
 - b. Versión paralela (usando todos los núcleos disponibles o un número configurable de núcleos).
3. **Controlar explícitamente el número de hilos/núcleos usados en la ejecución paralela.**
4. **Medir y comparar los tiempos de ejecución, calculando speedup y eficiencia.**
5. **Presentar un informe con:**
 - a. Código fuente comentado.
 - b. Gráficas de tiempos vs. núcleos.
 - c. Análisis crítico del comportamiento (por qué ocurre el speedup, cuellos de botella, etc.).
 - d. Reflexión sobre aplicaciones reales de este tipo de procesamiento.

Recomendaciones Técnicas

- Utilizar **OpenMP** en C/C++ o **multiprocessing** en Python.
- Tomar los archivos de ADN (4MB, 4GB) del link en la plataforma EVA.
- Usar funciones para fijar la cantidad de hilos (`omp_set_num_threads()` en OpenMP, o argumento `processes` en multiprocessing).

Rúbrica de Evaluación (50 puntos en total)

Criterio	Descripción	Puntaje Máximo
1. Correcta implementación secuencial	La versión secuencial funciona correctamente y procesa el archivo completo sin errores.	10
2. Implementación paralela funcional y configurable	La versión paralela usa múltiples núcleos (cantidad configurable) y logra dividir el trabajo.	10
3. Medición y análisis de desempeño	Incluye medición de tiempos, speedup, eficiencia, gráficas claras y análisis bien fundamentado.	10
4. Control y manejo de recursos de CPU	El número de núcleos se controla explícitamente y el programa maneja correctamente los hilos.	10
5. Presentación y documentación	Informe bien estructurado, código comentado, reflexiones, aplicaciones y bibliografía citada.	10
Total		50

P1.2: Computación GPU Paralela



Proyecto: Comparación del Rendimiento entre CPU y GPU en el Procesamiento Paralelo de Cadenas de ADN

Desarrollar una aplicación que procese un archivo de 4GB que contenga una cadena de ADN, aplicando técnicas de computación paralela sobre **GPU multinúcleo**, y permita comparar el rendimiento contra una ejecución equivalente en **CPU multihilo**.

Requerimientos Técnicos

Entrada:

- Archivo de 4GB simulando una cadena de ADN (caracteres A, C, T, G y posibles errores).
- Debe contener caracteres válidos y no válidos para análisis.

Funcionalidades:

1. **Procesamiento paralelo en CPU** utilizando hilos controlados (por ejemplo, con OpenMP o threads en C++/Python).
2. **Procesamiento paralelo en GPU**, utilizando CUDA o librerías como CuPy o Numba para Python.
3. **Identificación de caracteres inválidos** en la cadena.
4. **Comparación de desempeño:**
 - Tiempo total de procesamiento.
 - Uso de CPU/GPU (memoria, carga).
 - Gráficas comparativas (opcional con matplotlib).

Criterios de Comparación

- Tiempo de ejecución total.
- Utilización de recursos de hardware (GPU vs. CPU).
- Velocidad de identificación de errores por segundo.
- Escalabilidad ante tamaños crecientes.
- Visualización gráfica de resultados.

Tecnologías Recomendadas

Plataforma	Lenguaje	Librerías
CPU	C++/Python	OpenMP, Thread, multiprocessing
GPU	Python	Numba, CuPy, PyCUDA, TensorRT
Visualización	Python	matplotlib, seaborn

Rúbrica de Evaluación (50 puntos)

Criterio	Puntaje Máximo
1. Funcionamiento correcto del procesamiento CPU	10
2. Funcionamiento correcto del procesamiento GPU	10
3. Correcta identificación de caracteres inválidos	10
4. Medición y análisis del desempeño comparativo	10
5. Presentación clara del informe y visualizaciones	10
Total	50 puntos

Puntos adicionales:

- Benchmark sobre diferentes GPUs (discreta vs integrada).
- Escenarios con diferentes tipos de errores.
- Ajuste de batch size para balance de carga GPU.

P1. 3 computación CPU & GPU Paralela

CPU y GPU trabajando en paralelo.



OBJETIVOS DE PROYECTO:

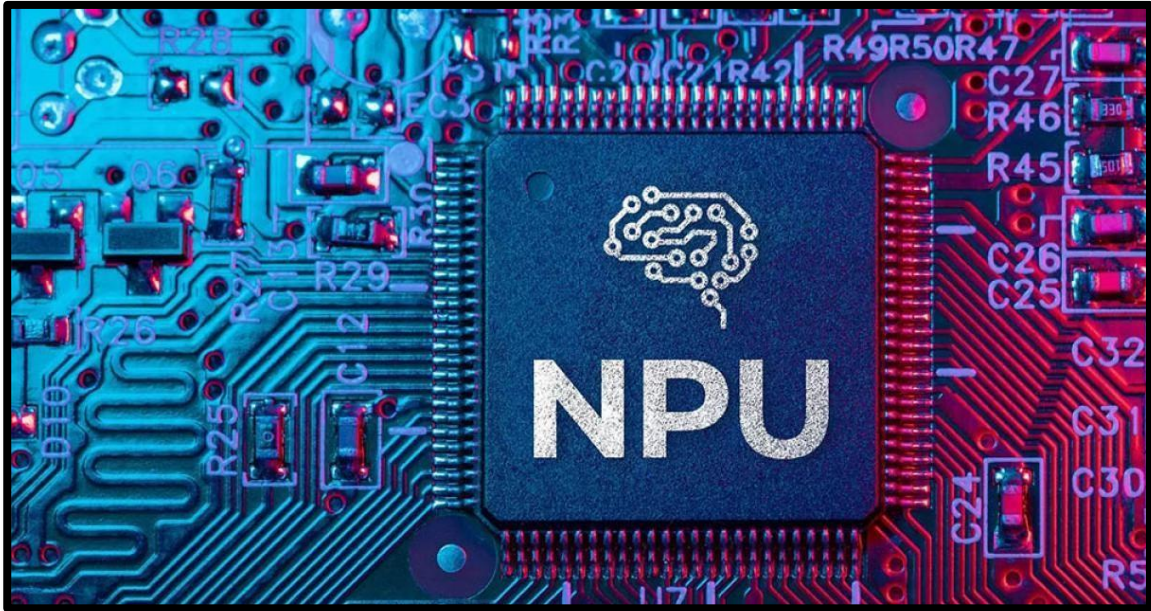
1. Recordar los componentes de un computador.
2. Diferenciar los propósitos de un CPU vs GPU.
3. Diferencias los propósitos de un computador personal vs un servidor empresarial.
4. Resumir las características de los supercomputadores.
5. Resumir los propósitos de los supercomputadores.

RUBRICA:

CRITERIO	No Presenta 0	CPU	GPU	GPU x núcleos
1) PROGRAMA	No presenta.	Presenta el código para CPU	Presenta el código para GPU	Presenta el código para GPU x núcleo.
2) LIBRERIA	No presenta.	Presenta la LIB que permite usar CPU.	Presenta la LIB que permite usar GPU.	Presenta la LIB que permite usar núcleos de GPU.
3) INSTRUCCION	No presenta.	Presenta la INS que permite usar CPU.	Presenta la INS que permite usar GPU.	Presenta la INS que permite usar núcleos de GPU.
4) % USO X NUCLEO	No presenta.	Presenta el % USO de CPU.	Presenta el % USO de GPU	Presenta el % USO de núcleos de GPU.
5) Temperatura x NUCLEO.	No presenta	Presenta la temperatura en CELCIUS de CPU.	Presenta la temperatura en CELCIUS de GPU.	Presenta la temperatura en CELCIUS de núcleos de GPU.
6) % USO de RAM	No presenta.	Presenta el % USO de RAM con CPU.	Presenta el % USO de RAM con GPU	Presenta el % USO de RAM con núcleos de GPU.

P1.4 Computación NPU Paralela

NPU procesamiento paralelo?

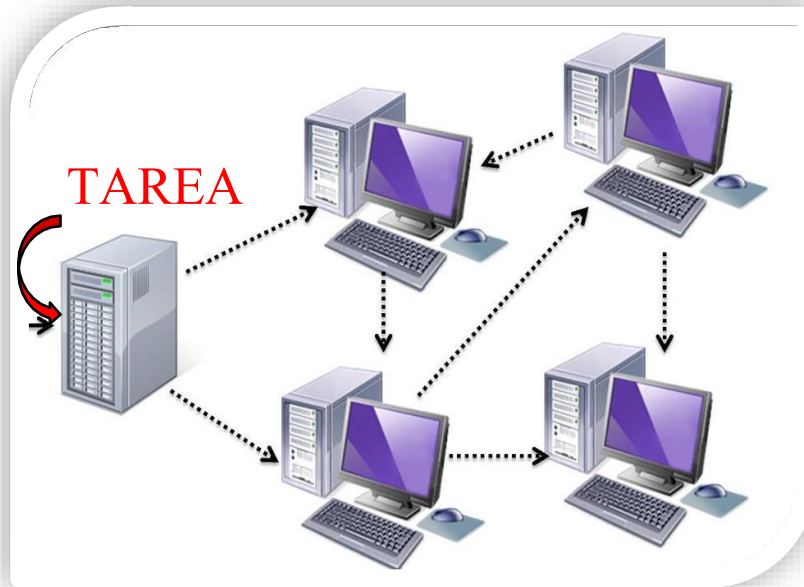


OBJETIVOS DE PROYECTO:

1. Recordar los componentes de un computador.
2. Diferenciar los propósitos de un CPU vs GPU vs NPU
3. Diferencias los propósitos de un computador personal vs un servidor empresarial.
4. Resumir las características de los supercomputadores.
5. Resumir los propósitos de los supercomputadores.

RUBRICA:

CRITERIO	No Presenta 0	CPU	GPU	NPU
1) PROGRAMA	No presenta.	Presenta el código para CPU	Presenta el código para GPU	Presenta el código para NPU x núcleo.
2) LIBRERIA	No presenta.	Presenta la LIB que permite usar CPU.	Presenta la LIB que permite usar GPU.	Presenta la LIB que permite usar núcleos de NPU.
3) INSTRUCCION	No presenta.	Presenta la INS que permite usar CPU.	Presenta la INS que permite usar GPU.	Presenta la INS que permite usar núcleos de NPU.
4) % USO X NUCLEO	No presenta.	Presenta el % USO de CPU.	Presenta el % USO de GPU	Presenta el % USO de núcleos de NPU.
5) Temperatura x NUCLEO.	No presenta	Presenta la temperatura en CELCIUS de CPU.	Presenta la temperatura en CELCIUS de GPU.	Presenta la temperatura en CELCIUS de núcleos de NPU.
6) % USO de RAM	No presenta.	Presenta el % USO de RAM con CPU.	Presenta el % USO de RAM con GPU	Presenta el % USO de RAM con núcleos de NPU.



COMPUTACION DISTRIBUIDA

II.- DISTRIBUTED COMPUTING

P2.1 Clúster de Computadoras

El término clúster (del inglés clúster, que significa 'grupo' o 'racimo') se aplica a los sistemas distribuidos de granjas de computadoras unidos entre sí normalmente por una red de alta velocidad y que se comportan como si fuesen un único servidor.

A diferencia de la computación en malla (computación GRID), los clústeres de computadoras tienen a cada nodo realizando la misma tarea, controlada y planificada por software.

Referencias:

https://es.wikipedia.org/wiki/Cl%C3%Baster_de_computadoras

<https://www.youtube.com/watch?v=SCrhrcYEhW8>

<https://www.youtube.com/watch?v=oQeURfJCg-g>



File:Us-nasa-columbia.jpg

CONSIDERACIONES Y SUGERENCIAS:

- Utilizar tres [3] máquinas virtuales [nodos x participante]
- Utilizar tres [3] máquinas reales [nodos x grupo]
- Utilizar un sistema operativo tipo CLÚSTER.
- Revisar los comandos para verificar las conexiones entre nodos
- Revisar la estructura de nodo raíz y nodos de clúster.

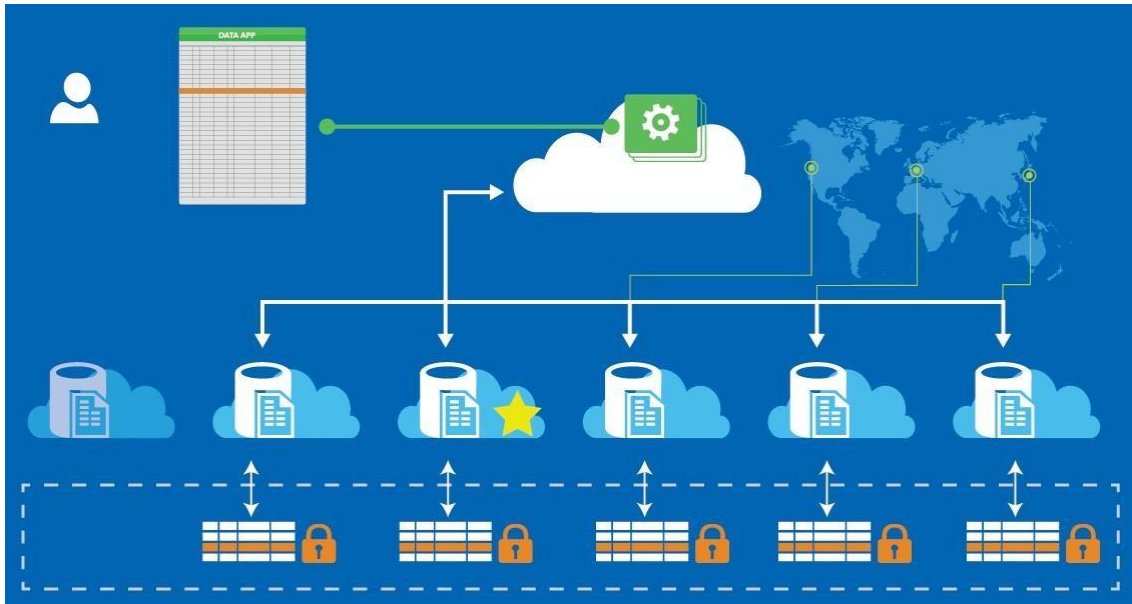
OBJETIVOS DE PROYECTO:

1. Implementar un clúster utilizando máquinas virtuales.
2. Implementar un clúster utilizando máquinas reales.
3. Verificar sistemas operativos distribuidos.
4. Verificar sistemas operativos para HPC.
5. Verificar sistema operativo para HA.

RUBRICA:

CRITERIO	No Presenta 0	BAJO 3	MEDIO 6	ALTO 10
1. clúster con máquinas VIRTUALES.	No presenta.	Presenta el diseño gráfico del CLÚSTER.	Explica la conectividad y direcciones IP.	Presenta funcionando el CLÚSTER.
3. Sistema operativo en clúster virtual.	No presenta.	Presenta el funcionamiento de un nodo.	Presenta el funcionamiento de dos nodos.	Presenta el funcionamiento del clúster desde el nodo máster.
3. clúster con máquinas REALES.	No presenta.	Presenta el diseño gráfico del CLÚSTER.	Explica la conectividad y direcciones IP.	Presenta funcionando el CLÚSTER.
4. Sistema operativo en clúster virtual.	No presenta.	Presenta el funcionamiento de un nodo.	Presenta el funcionamiento de dos nodos.	Presenta el funcionamiento del clúster desde el nodo máster.
5. Recomendaciones	No presenta.	Presenta una recomendación.	Presenta tres recomendaciones.	Presenta cinco recomendaciones.

P2.2 Procesamiento Distribuido



Proyecto: Procesamiento Distribuido de Cadenas de ADN en Clúster

Objetivo:

Diseñar e implementar un sistema de computación distribuida para analizar y procesar una cadena de ADN de 1GB, distribuyendo la carga de trabajo entre varios nodos personales conectados en red (clúster local).

Descripción Técnica

Tipo de arquitectura: Clúster homogéneo con Linux o WSL (Windows Subsystem for Linux).

Lenguaje sugerido: Python con librería Ray o mpi4py.

Número de nodos: 1 nodo maestro + 3–4 nodos esclavos (PCs personales).

Tareas distribuidas:

Fragmentación del archivo de ADN.

Conteo de bases (A, T, C, G).

Búsqueda de patrones genéticos específicos.

Reporte y consolidación de resultados.

Actividades clave del proyecto:

Dividir el archivo de ADN (1GB) en fragmentos que puedan ser distribuidos.

Diseñar la lógica de procesamiento distribuido, aplicando patrones de computación distribuida.

Configurar el clúster de computadoras con herramientas como Ray, MPI, o BOINC.

Ejecutar y medir el procesamiento paralelo.

Comparar resultados vs. procesamiento secuencial.

Resultados esperados

Reducción del tiempo de procesamiento.

Validación de la integridad de los resultados entre nodos.

Evaluación de eficiencia, escalabilidad y tolerancia a fallos.

Rúbrica de Evaluación (50 puntos)

Criterio	Descripción	Puntaje Máximo
1. Diseño de Arquitectura del Clúster	Topología, configuración de red, asignación de nodos y justificación técnica.	10
2. División y Distribución del Trabajo	Fragmentación eficiente del archivo de ADN, asignación y sincronización.	10
3. Implementación Técnica Funcional	Uso correcto de librerías (Ray, mpi4py, etc.), ejecución estable y completa.	10
4. Resultados y Validación	Precisión en conteo/patrón, comparación entre nodos, consolidación correcta.	10
5. Documentación y Presentación	Informe técnico claro, reflexión sobre problemas enfrentados, presentación oral.	10

Recomendaciones

Utilizar archivos FASTA o similares para pruebas.

Aprovechar ciclos ociosos y simular fallos para probar tolerancia.

Incorporar métricas como *speedup* y *eficiencia* para analizar escalabilidad.

P2.3 Procesamiento Distribuido Optimizado

El objetivo final es que el estudiante construya un sistema capaz de orquestar diferentes tipos de hardware en una red de computadores personales, seleccionando dinámicamente el mejor recurso para cada tarea.

Proyecto Integrador: "Ecosistema Genómico Heterogéneo y Distribuido"

1. Arquitectura del Sistema

El proyecto se divide en tres capas funcionales que se irán completando a lo largo de las 3 semanas:

- **Capa de Orquestación (Master):** Un nodo (laptop del líder) que divide el archivo de ADN, gestiona la red (NFS/SSH) y asigna bloques de datos según la potencia de cada nodo trabajador.
- **Capa de Cómputo (Workers):** Una red de computadores personales que ofrecen sus recursos (CPU, GPU o NPU) al clúster.
- **Capa de Análisis (Dashboard):** Una interfaz que muestra en tiempo real qué arquitectura está procesando los datos y cuál es su eficiencia (Tiempo vs. Energía).

2. Cronograma de Integración Semanal

Semana 1: El Motor de Control (CPU & Red)

- **Tarea:** Configurar el clúster físico y el software base.
- **Meta:** Ejecutar la comparación de ADN usando el 100% de los núcleos de todas las CPUs del clúster mediante **MPI**.
- **Foco Técnico:** Afinidad de procesos y minimización de latencia en la red Ethernet.

Semana 2: Aceleración de Fuerza Bruta (GPU)

- **Tarea:** Integrar la capacidad de cómputo gráfico de las laptops que cuenten con GPU dedicada.
- **Meta:** El Master detecta qué nodos tienen GPU y les envía los fragmentos más pesados del archivo para procesamiento masivo en **CUDA/OpenCL**.
- **Foco Técnico:** Solapamiento de transferencia de datos (PCIe) con el cálculo.

Semana 3: Inteligencia y Optimización (NPU)

- **Tarea:** Implementar modelos de clasificación o búsqueda de motivos complejos.
- **Meta:** Utilizar las NPUs de los procesadores modernos para detectar "zonas de interés" en la cadena de ADN de forma ultra-eficiente.
- **Foco Técnico:** Cuantización INT8 y eficiencia energética (TOPS/Watt).

4. Rúbrica de Evaluación Integral

Categoría	Criterio de Evaluación	Puntaje
Infraestructura (Red)	Estabilidad del clúster bajo carga máxima y configuración correcta del sistema de archivos distribuido (NFS).	20
Optimización CPU	Uso correcto de instrucciones vectoriales (SIMD) y control total de la afinidad de núcleos.	20

Eficiencia GPU/NPU	Capacidad de transferir datos a la VRAM sin bloquear la CPU y uso de modelos cuantizados en la NPU.	20
Balanceo de Carga	El nodo Master asigna tareas de forma inteligente (más trabajo al nodo más rápido) evitando que el sistema se ralentice por el "nodo más lento".	20
Informe Técnico	Análisis comparativo final: ¿Qué arquitectura fue más rentable (Tiempo/Costo/Energía)? Uso de Ley de Amdahl.	20

5. Preparación del Entorno (Checklist):

1. **Conexión Física:** Switch Gigabit y cables Ethernet (prohibido Wi-Fi para la prueba final).
2. **Sistema Base:** Linux (Ubuntu/Debian) preferiblemente para facilitar la configuración de MPI y drivers.
3. **Software:** * OpenMPI para la comunicación distribuida.
 - o GCC con soporte AVX.
 - o NVIDIA CUDA Toolkit y OpenVINO/TensorRT.

6. Guía de Comandos UNIX

Comandos LINUX

EXAMEN Distributed Computing:

Se entregará un **archivo de 10GB de ADN**.

El equipo ganador no será necesariamente el que tenga las mejores laptops, sino el que mejor haya **balanceado la carga**: aquel que logre que el i3 de un compañero trabaje en armonía con el i9 con GPU de otro, manteniendo todos los procesadores al 100% de su capacidad individual.



COMPUTACION EN LA NUBE

III.- CLOUD COMPUTING

P 3.1: XaaS Proveedores AMERICANOS

Proveedor	Sede Principal	Servicios Destacados	Fortalezas	Casos de Uso Comunes
Amazon Web Services (AWS)	Seattle, Washington	EC2, S3, RDS, Lambda, SageMaker	Líder del mercado, gran cantidad de servicios, disponibilidad global	Empresas grandes, startups, servicios globales
Microsoft Azure	Redmond, Washington	Azure VMs, Azure Functions, Cosmos DB, Azure AI	Integración con productos Microsoft, fuerte en sector empresarial	Organizaciones que usan Windows y Office
Google Cloud Platform (GCP)	Mountain View, California	Compute Engine, BigQuery, Cloud Functions, Vertex AI	Excelente en datos, IA y aprendizaje automático	Ciencia de datos, IA, análisis en tiempo real
IBM Cloud	Armonk, Nueva York	IBM Cloud Functions, Watson AI, Cloud Kubernetes	Soluciones híbridas, fuerte en inteligencia artificial con Watson	Finanzas, salud, sector público
Oracle Cloud	Austin, Texas	Oracle Autonomous DB, Oracle Functions, Compute Infrastructure	Bases de datos empresariales, optimizado para sistemas Oracle	ERP, bases de datos, aplicaciones empresariales
DigitalOcean	Nueva York, Nueva York	Droplets, App Platform, Managed Databases, Functions	Simplicidad, precios accesibles, orientado a desarrolladores	Startups, pymes, desarrolladores independientes
Linode (ahora Akamai)	Filadelfia, Pensilvania	Linode Instances, Kubernetes, Block Storage	Alto rendimiento a bajo costo, fácil de usar	Proyectos medianos, desarrolladores y sysadmins
Heroku (propiedad de Salesforce)	San Francisco, California	Dynos, Heroku Postgres, Pipelines	Facilidad de despliegue, abstracción de infraestructura	Aplicaciones web, prototipos, MVPs
Rackspace Technology	San Antonio, Texas	Gestión multicloud (AWS, Azure, GCP), seguridad, migración	Servicios gestionados, soporte 24/7, enfoque empresarial	Outsourcing de infraestructura, multicloud empresarial

CONSIDERACIONES:

1. Cada estudiante del grupo tiene un Proveedor asignado.
2. Ejecutar la problemática de cadenas de ADN – directamente en el proveedor seleccionado.
3. Evaluar los proveedores de Cloud Computing americanos.
4. Profundizar el concepto “CONTENEDORES” para llevar el programa de un proveedor a otro sin mayor dificultad.

RUBRICA:

Evaluar la **capacidad del estudiante o grupo** para aplicar conceptos de **contenedores y portabilidad** mediante herramientas como Docker para **migrar aplicaciones** entre entornos de distintos proveedores cloud (por ejemplo: de AWS a Azure, o de GCP a DigitalOcean).

Criterio	Excelente (10 pts)	Bueno (7-9 pts)	Regular (4-6 pts)	Deficiente (1-3 pts)
Aplicación en Contenedor (Docker o equivalente)	Aplica correctamente Docker (o similar), con Dockerfile, imagen funcional y uso de docker-compose.	Usa Docker con una imagen funcional, pero con documentación o estructura incompleta.	Docker se aplica de forma parcial, con errores o sin compose.	No usa contenedores correctamente o no funciona el despliegue.
Portabilidad entre proveedores Cloud	Demuestra despliegue exitoso y documentado en al menos dos proveedores diferentes.	Despliega en dos nubes, pero con problemas menores o sin justificar decisiones.	Solo despliega en un proveedor o con errores al mover el contenedor.	No hay evidencia funcional de portabilidad entre proveedores.
Automatización del despliegue (CI/CD o scripts)	Utiliza pipelines o scripts (Docker, Bash, YAML) para automatizar la migración y despliegue.	Usa scripts simples o parciales sin automatización completa.	Scripts incompletos, manualidades excesivas.	No hay automatización ni intento claro de scripting.
Análisis comparativo de proveedores Cloud (en contexto Docker)	Compara servicios, soporte de contenedores, redes, costos y facilidad de integración.	Presenta comparación, pero no profundiza en todos los aspectos técnicos relevantes.	Menciona solo diferencias generales sin relación con contenedores.	No analiza diferencias o solo cita información sin relación técnica.
Documentación técnica del proceso	Incluye README, estructura de carpetas, pasos claros para ejecución y migración reproducible.	Incluye documentación parcial o desactualizada.	Documentación básica sin claridad total en el proceso.	No hay documentación o es confusa e incompleta.

P 3.2 XaaS Proveedores ASIATICOS

A continuación, un cuadro resumen de los proveedores asiáticos y los servicios que ofrecen tipo XaaS.

Proveedor	País de Origen	Servicios Destacados	Fortalezas	Casos de Uso Comunes
Alibaba Cloud	China	Elastic Compute Service (ECS), ApsaraDB, Function Compute (FaaS)	Segundo proveedor mundial después de AWS en algunas regiones de Asia; fuerte presencia en China y Sudeste Asiático	E-commerce, IA, big data, empresas asiáticas
Tencent Cloud	China	CVM (Cloud Virtual Machine), SCF (Serverless Cloud Function), TDSQL	Potente para gaming, redes sociales, gran capacidad de escalamiento	Juegos, aplicaciones móviles, entretenimiento
Huawei Cloud	China	Elastic Cloud Server (ECS), ModelArts (IA), FunctionGraph (FaaS)	Orientado a IA, telecomunicaciones y gobiernos; fuerte en nube híbrida	Telecomunicaciones, industria, smart cities
NTT Communications	Japón	Enterprise Cloud, Cloudn, Global Cloud Platform	Infraestructura empresarial sólida, compliance y seguridad para sector financiero	Finanzas, grandes corporaciones en Japón y Asia
NHN Cloud (ex-Naver Cloud)	Corea del Sur	Compute, Kubernetes, AI Services, Cloud DB	Enfocado en contenido digital, videojuegos, medios y startups	Medios digitales, ecommerce, servicios coreanos
Sakura Internet	Japón	VPS, IaaS, GPU Cloud, Sakura Cloud	Asequible y fácil de usar para desarrolladores locales	Startups, desarrolladores japoneses, PYMEs
KT Cloud	Corea del Sur	KT Cloud Platform, AI Cloud, Cloud Containers	Integración con 5G, IoT, y servicios públicos	Gobiernos, telecomunicaciones, infraestructura

CONSIDERACIONES:

1. Cada estudiante del grupo tiene un Proveedor asignado.
2. Ejecutar la problemática de cadenas de ADN – directamente en el proveedor seleccionado.
3. Evaluar los proveedores de Cloud Computing.
4. Profundizar el concepto “CONTENEDORES” para llevar el programa de un proveedor a otro sin mayor dificultad.

RUBRICA:

Criterio	Excelente (10 pts)	Bueno (7-9 pts)	Regular (4-6 pts)	Deficiente (1-3 pts)
Aplicación en Contenedor (Docker o equivalente)	Aplica correctamente Docker (o similar), con Dockerfile, imagen funcional y uso de docker-compose.	Usa Docker con una imagen funcional, pero con documentación o estructura incompleta.	Docker se aplica de forma parcial, con errores o sin compose.	No usa contenedores correctamente o no funciona el despliegue.
Portabilidad entre proveedores Cloud	Demuestra despliegue exitoso y documentado en al menos dos proveedores diferentes.	Despliega en dos nubes, pero con problemas menores o sin justificar decisiones.	Solo despliega en un proveedor o con errores al mover el contenedor.	No hay evidencia funcional de portabilidad entre proveedores.
Automatización del despliegue (CI/CD o scripts)	Utiliza pipelines o scripts (Docker, Bash, YAML) para automatizar la migración y despliegue.	Usa scripts simples o parciales sin automatización completa.	Scripts incompletos, manualidades excesivas.	No hay automatización ni intento claro de scripting.
Análisis comparativo de proveedores Cloud (en contexto Docker)	Compara servicios, soporte de contenedores, redes, costos y facilidad de integración.	Presenta comparación, pero no profundiza en todos los aspectos técnicos relevantes.	Menciona solo diferencias generales sin relación con contenedores.	No analiza diferencias o solo cita información sin relación técnica.
Documentación técnica del proceso	Incluye README, estructura de carpetas, pasos claros para ejecución y migración reproducible.	Incluye documentación parcial o desactualizada.	Documentación básica sin claridad total en el proceso.	No hay documentación o es confusa e incompleta.

P 3.3 XaaS Proveedores EUROPEOS

Proveedor	País de Origen	Servicios Cloud Destacados	Fortalezas	Casos de Uso Comunes
OVHcloud	Francia	Public Cloud, Private Cloud, Bare Metal, Kubernetes, Object Storage	Centros de datos propios en Europa, cumplimiento GDPR, precios competitivos	Hosting, SaaS europeos, startups
Deutsche Telekom (T-Systems)	Alemania	Open Telekom Cloud (IaaS, PaaS, backup, SAP Cloud)	Seguridad, cumplimiento GDPR, fuerte en sector público	Empresas alemanas, instituciones públicas, SAP
Scaleway	Francia	Instances, Serverless, Managed Kubernetes, Object Storage	Innovador, enfoque en sostenibilidad y eficiencia energética	Startups, proyectos verdes, desarrolladores
Atos Cloud	Francia	Canopy Cloud, infraestructura híbrida, servicios TI, HPC	Consultoría especializada, foco en HPC y empresas reguladas	Defensa, salud, gobierno, banca
IONOS (by 1&1)	Alemania	VPS, Cloud Server, IaaS, Kubernetes, soluciones para PyMEs	Infraestructura europea, atención a PyMEs, panel simple	Web hosting, PyMEs, ecommerce
CloudSigma	Suiza	Infraestructura flexible (CPU/RAM a medida), redes privadas, almacenamiento SSD	Personalización completa de infraestructura, foco en cumplimiento	Instituciones financieras, proyectos sensibles
Swisscom Cloud	Suiza	IaaS, PaaS, Kubernetes, DevOps tools, servicios multicloud	Privacidad, cumplimiento normativo suizo y europeo	Finanzas, salud, telecomunicaciones
Exoscale	Suiza	Compute, S3-compatible Storage, Kubernetes, FaaS	Simplicidad, precios transparentes, data centers europeos	Startups, desarrolladores, científicos de datos
Orange Cloud	Francia	Nube híbrida, IaaS/PaaS, almacenamiento, Big Data, seguridad cloud	Infraestructura confiable, integración con telecomunicaciones	Empresas francesas,

CONSIDERACIONES:

1. Cada estudiante del grupo tiene un Proveedor asignado.
2. Ejecutar la problemática de cadenas de ADN – directamente en el proveedor seleccionado.
3. Evaluar los proveedores de Cloud Computing.
4. Profundizar el concepto “CONTENEDORES” para llevar el programa de un proveedor a otro sin mayor dificultad.

RUBRICA:

Criterio	Excelente (10 pts)	Bueno (7-9 pts)	Regular (4-6 pts)	Deficiente (1-3 pts)
Aplicación en Contenedor (Docker o equivalente)	Aplica correctamente Docker (o similar), con Dockerfile, imagen funcional y uso de docker-compose.	Usa Docker con una imagen funcional, pero con documentación o estructura incompleta.	Docker se aplica de forma parcial, con errores o sin compose.	No usa contenedores correctamente o no funciona el despliegue.

Portabilidad entre proveedores Cloud	Demuestra despliegue exitoso y documentado en al menos dos proveedores diferentes.	Despliega en dos nubes, pero con problemas menores o sin justificar decisiones.	Solo despliega en un proveedor o con errores al mover el contenedor.	No hay evidencia funcional de portabilidad entre proveedores.
Automatización del despliegue (CI/CD o scripts)	Utiliza pipelines o scripts (Docker, Bash, YAML) para automatizar la migración y despliegue.	Usa scripts simples o parciales sin automatización completa.	Scripts incompletos, manualidades excesivas.	No hay automatización ni intento claro de scripting.
Análisis comparativo de proveedores Cloud (en contexto Docker)	Compara servicios, soporte de contenedores, redes, costos y facilidad de integración.	Presenta comparación, pero no profundiza en todos los aspectos técnicos relevantes.	Menciona solo diferencias generales sin relación con contenedores.	No analiza diferencias o solo cita información sin relación técnica.
Documentación técnica del proceso	Incluye README, estructura de carpetas, pasos claros para ejecución y migración reproducible.	Incluye documentación parcial o desactualizada.	Documentación básica sin claridad total en el proceso.	No hay documentación o es confusa e incompleta.

P 3.4 XaaS Proveedores ECUATORIANOS

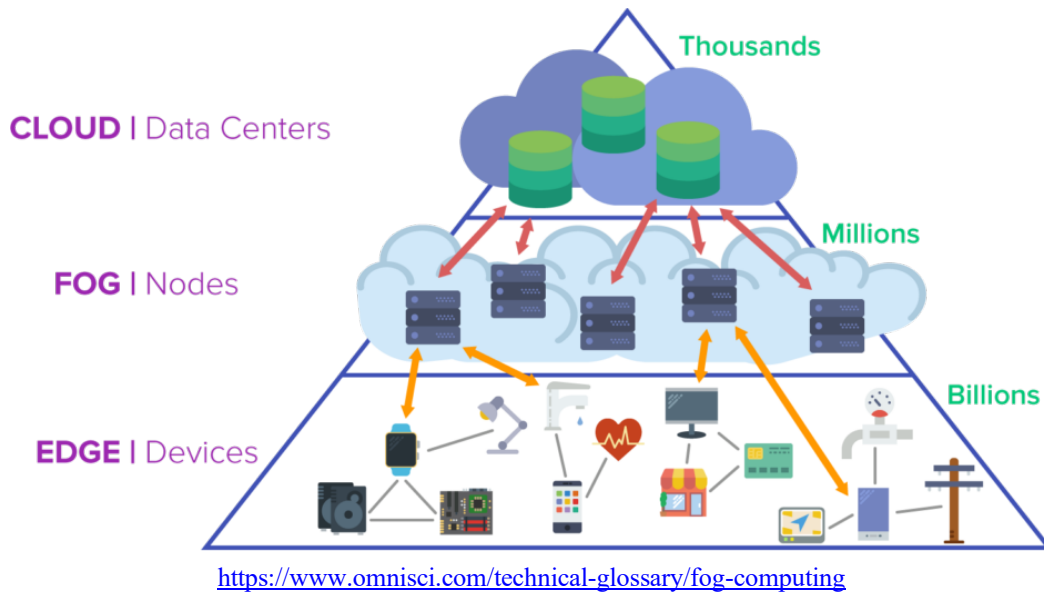
Proveedor	Ciudad/Sede	Servicios Cloud Destacados	Fortalezas	Casos de Uso Comunes
Telconet	Guayaquil / Quito	Cloud pública y privada, housing, backup, disaster recovery, servidores virtuales	Infraestructura nacional robusta, centro de datos TIER III certificado	Banca, gobierno, ISPs, grandes empresas
Netlife (PuntoNet)	Quito / Guayaquil	Hosting, servidores cloud, soluciones VPS, conectividad empresarial	Excelente conectividad nacional, soluciones básicas de cloud	Emprendedores, PyMEs, educación
CNT EP	Quito	Cloud pública estatal, correo, hosting, backup, servidores virtuales	Empresa estatal, cobertura nacional, foco en sector público	Educación, instituciones del Estado, salud
Claro Cloud	Quito / Guayaquil	IaaS, almacenamiento, correo, servidores VPS, backup	Infraestructura de América Móvil, integración con servicios móviles y fijos	Corporaciones, telecomunicaciones, PyMEs
Movistar Empresas	Quito / Guayaquil	Infraestructura gestionada, nube híbrida, FaaS, respaldo, email corporativo	Alianzas con Azure y Google Cloud, buen soporte TI	Educación, banca, transformación digital empresarial
Sonda Ecuador	Quito / Guayaquil	Cloud híbrida, infraestructura SAP en la nube, servicios TI gestionados	Foco en sector empresarial, integración de sistemas complejos	ERP, SAP, gobierno, banca
Internexa Ecuador	Quito / Guayaquil	IaaS, housing, nube privada, interconexión regional	Red propia regional, infraestructura para clientes multinacionales	Telecomunicaciones, carriers, banca
Digiware Ecuador	Quito	Ciberseguridad en la nube, SOC, cumplimiento normativo cloud	Especialistas en protección de datos y amenazas avanzadas	Infraestructura crítica, sector salud y financiero
Grupo Linke	Quito	Cloud privada, respaldo, consultoría TI, soluciones Microsoft Cloud	Partner de Microsoft, atención personalizada	PyMEs, proyectos con soluciones Microsoft
Andean (AndeanWire)	Quito	Hosting, VPS, correo empresarial, respaldo en la nube	Foco en atención a pequeñas empresas, rápida respuesta técnica	Hosting web, PyMEs, desarrolladores
Xtrim (TV Cable)	Quito / Guayaquil	VPS, hosting web, correo corporativo	Bajo costo y cobertura nacional en servicios básicos cloud	Emprendedores, proyectos académicos

CONSIDERACIONES:

1. Cada estudiante del grupo tiene un Proveedor asignado.
2. Ejecutar la problemática de cadenas de ADN – directamente en el proveedor seleccionado.
3. Evaluar los proveedores de Cloud Computing.
4. Profundizar el concepto “CONTENEDORES” para llevar el programa de un proveedor a otro sin mayor dificultad.

RUBRICA:

Criterio	Excelente (10 pts)	Bueno (7-9 pts)	Regular (4-6 pts)	Deficiente (1-3 pts)
Aplicación en Contenedor (Docker o equivalente)	Aplica correctamente Docker (o similar), con <code>Dockerfile</code> , imagen funcional y uso de <code>docker-compose</code> .	Usa Docker con una imagen funcional, pero con documentación o estructura incompleta.	Docker se aplica de forma parcial, con errores o sin <code>compose</code> .	No usa contenedores correctamente o no funciona el despliegue.
Portabilidad entre proveedores Cloud	Demuestra despliegue exitoso y documentado en al menos dos proveedores diferentes.	Despliega en dos nubes, pero con problemas menores o sin justificar decisiones.	Solo despliega en un proveedor o con errores al mover el contenedor.	No hay evidencia funcional de portabilidad entre proveedores.
Automatización del despliegue (CI/CD o scripts)	Utiliza pipelines o scripts (Docker, Bash, YAML) para automatizar la migración y despliegue.	Usa scripts simples o parciales sin automatización completa.	Scripts incompletos, manualidades excesivas.	No hay automatización ni intento claro de scripting.
Análisis comparativo de proveedores Cloud (en contexto Docker)	Compara servicios, soporte de contenedores, redes, costos y facilidad de integración.	Presenta comparación, pero no profundiza en todos los aspectos técnicos relevantes.	Menciona solo diferencias generales sin relación con contenedores.	No analiza diferencias o solo cita información sin relación técnica.
Documentación técnica del proceso	Incluye README, estructura de carpetas, pasos claros para ejecución y migración reproducible.	Incluye documentación parcial o desactualizada.	Documentación básica sin claridad total en el proceso.	No hay documentación o es confusa e incompleta.

IV.- NEXT COMPUTING

CLOUD → FOG →
EDGE

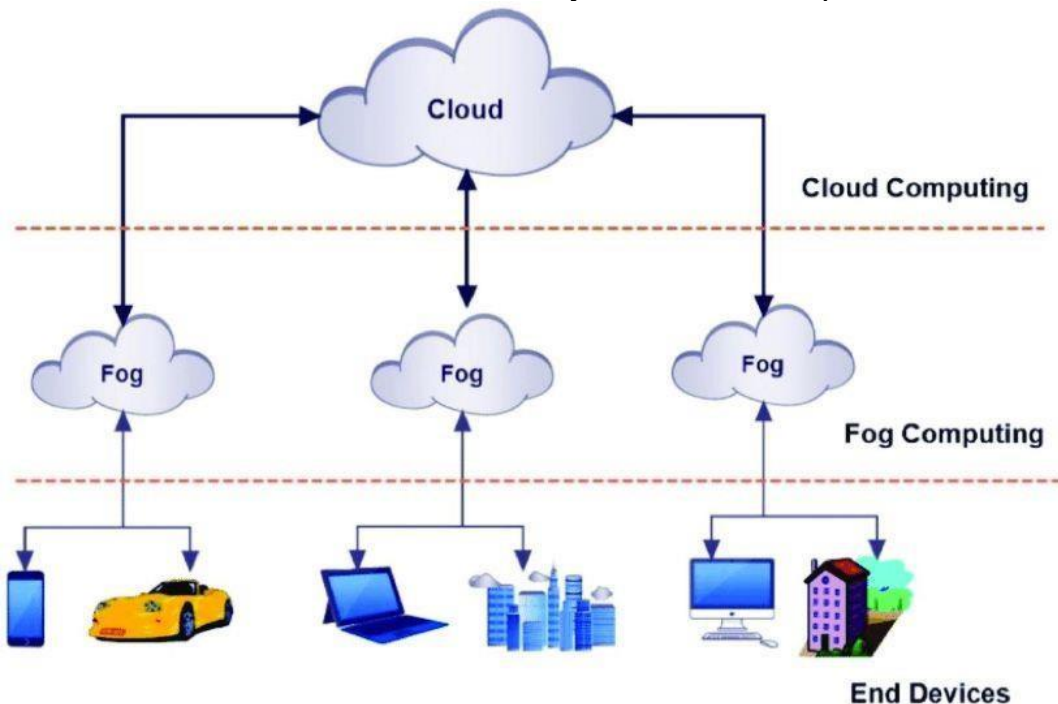
P 4.1 FOG Computing

Fog Computing, o **computación en la niebla**, es un paradigma de computación distribuida que extiende los servicios y recursos de la nube (Cloud) hacia el borde de la red (Edge), acercando el procesamiento, almacenamiento y análisis de datos a los dispositivos que generan dicha información (sensores, IoT, móviles, etc.).

Características principales de Fog Computing

1. **Procesamiento distribuido:** Reduce la latencia al procesar datos más cerca de la fuente.
2. **Soporte para IoT:** Ideal para escenarios con miles/millones de sensores (smart cities, fábricas inteligentes, vehículos autónomos).
3. **Seguridad y privacidad:** Permite aplicar políticas de seguridad cerca de la fuente de datos.
4. **Interoperabilidad:** Integra diferentes dispositivos, protocolos y arquitecturas.
5. **Escalabilidad:** Permite crecer en función del número de nodos fog según la demanda.

- Se reduce el consumo de ancho de banda y se obtiene una respuesta casi en



<https://www.winsystems.com/an-update-on-fog-computing/>

Proyecto: Análisis Distribuido de Secuencias de ADN usando Fog Computing

Desarrollar un prototipo funcional que permita analizar una cadena de ADN de 6 GB, utilizando una arquitectura tipo *fog computing*, es decir, distribuyendo la carga de procesamiento entre varias computadoras (nodos fog) en red local antes de consolidar resultados en un nodo central o en la nube. El sistema debe identificar patrones específicos (por ejemplo, secuencias génicas, repeticiones, mutaciones) y reportar los hallazgos principales.

Objetivos:

- Aplicar conceptos de computación distribuida y fog computing en el procesamiento de datos masivos.
- Utilizar técnicas de procesamiento paralelo (threads/procesos/hilos o frameworks tipo Apache Spark, Dask, etc.).
- Integrar el análisis bioinformático de cadenas de ADN (búsqueda de patrones).
- Implementar una solución funcional y evaluable en una red local universitaria.

Fases y Entregables

- Diseño de la Arquitectura
- Diagrama de la arquitectura fog (al menos 3 nodos fog y un nodo central).
- División y Distribución de Datos
- Algoritmo para dividir el archivo de 6GB entre nodos.
- Procesamiento en los Nodos Fog
- Desarrollo de un programa (Python, Java, etc.) para búsqueda de patrones o mutaciones en cada partición.

Consolidación de Resultados

- Recopilación y presentación de resultados en el nodo central (dashboard simple o reporte consolidado).
- Documentación Técnica
- Manual breve del usuario, instrucciones de despliegue y evidencia de pruebas (capturas de pantalla, logs).

Tecnologías y Herramientas Sugeridas

- Lenguaje: Python o Java (preferentemente por facilidad de manejo de archivos grandes y bibliotecas).
- Bibliotecas: PySpark, Dask, multiprocessing (Python); Java threads o frameworks similares.
- Red: Simulación de nodos en PCs/laptops del laboratorio conectados en red local.
- Visualización: Jupyter Notebook, Dash, o simplemente archivos CSV.

Rúbrica de Evaluación (Total: 50 puntos)

Criterio	Puntaje Máx.
1. Diseño de Arquitectura Fog - Claridad, lógica, diagrama y justificación de roles de cada nodo.	10
2. Distribución de Datos - Algoritmo eficiente y manejo adecuado de archivos grandes.	8
3. Programación del Análisis - Calidad del código, uso de paralelismo/distribución y correcta detección de patrones.	12
4. Consolidación y Visualización - Presentación clara y entendible de resultados consolidados.	8
5. Documentación y Evidencias - Manuales, instrucciones y registro de pruebas funcionales.	7
6. Presentación Final - Capacidad de explicar la solución y responder preguntas técnicas.	5

P 4.2 EDGE Computing

Edge Computing (computación en el borde) es un paradigma donde el procesamiento de datos se realiza cerca del lugar donde se generan los datos, es decir, en dispositivos o servidores locales (“en el borde” de la red), en lugar de depender totalmente de un data center central o de la nube.

Características clave:

- **Descentralización:** El procesamiento ocurre en dispositivos locales (gateways, routers inteligentes, sensores, cámaras, etc.), no solo en la nube.
- **Baja latencia:** Al estar cerca del origen de los datos, se reduce el tiempo de respuesta.
- **Eficiencia de ancho de banda:** Se procesan y filtran los datos localmente, enviando solo información relevante a la nube.
- **Mejora de la privacidad:** Menos datos sensibles viajan por la red, lo que ayuda a cumplir normas de protección de datos.
- **Escalabilidad:** Permite gestionar grandes volúmenes de datos generados por dispositivos IoT sin sobrecargar la nube.

Diferencias respecto a Cloud Computing

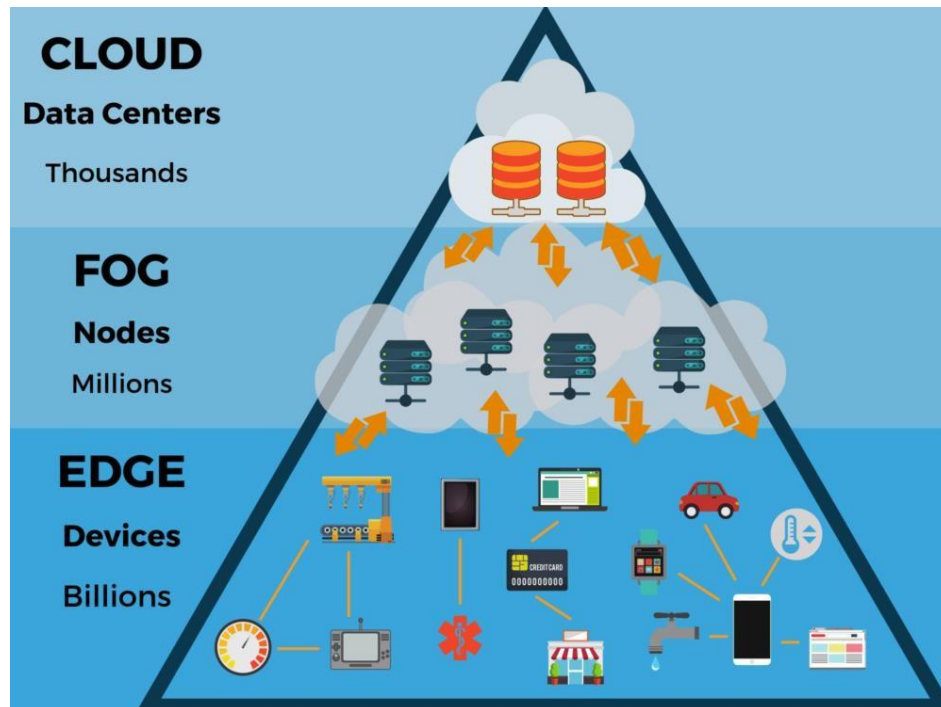
	Cloud Computing	Edge Computing
Procesamiento	En servidores centralizados	En dispositivos locales
Latencia	Mayor, por distancia	Muy baja, está “en el borde”
Privacidad	Datos viajan a la nube	Datos procesados localmente
Costo de transmisión	Alto, por envío masivo de datos	Bajo, por procesamiento local
Uso típico	Big Data, almacenamiento, IA pesada	IoT, video en tiempo real, aplicaciones críticas

Ejemplos de uso de EDGE Computing

1. **Vehículos autónomos:** El auto procesa datos de sensores en tiempo real sin depender de la nube para evitar accidentes.
2. **Cámaras de vigilancia inteligente:** Procesan imágenes localmente y solo envían alertas o clips relevantes.
3. **IoT en salud:** Un wearable monitorea signos vitales y solo notifica al sistema central si detecta anomalías.

Tendencias actuales

- **AI en el borde (Edge AI):** Algoritmos de IA corriendo localmente, por ejemplo, para reconocimiento facial en teléfonos móviles.
- **5G + Edge:** 5G potencia el despliegue de aplicaciones EDGE por su baja latencia y ancho de banda.
- **Fog Computing:** Un paso intermedio entre la nube y el EDGE, donde hay una capa de servidores locales (fog nodes) que agregan procesamiento distribuido.



<https://www.muycomputerpro.com/2019/06/10/edge-computing-casi-todo-lo-que-tenes-que-saber>

Proyecto: Análisis de Cadena de ADN en el Borde (EDGE Computing)

Desarrollar un prototipo que procese una cadena de ADN de 6 GB utilizando una arquitectura basada en EDGE Computing. El objetivo es demostrar cómo dividir, filtrar y analizar localmente grandes volúmenes de datos biomédicos en dispositivos edge antes de enviarlos a la nube para análisis avanzado o almacenamiento.

Objetivos de Aprendizaje:

- Aplicar conceptos de EDGE Computing para resolver un problema real de procesamiento masivo de datos.
- Implementar procesamiento distribuido y filtrado de información en el borde.
- Optimizar el uso del ancho de banda y minimizar la latencia en el análisis de datos biomédicos.
- Desarrollar competencias en integración de sistemas y seguridad de datos en arquitecturas EDGE.

Actividades y Entregables:

1. Investigación y Diseño (Día 1):

- Breve investigación sobre arquitecturas EDGE en bioinformática.

Diseño del flujo de procesamiento y selección de herramientas (ejemplo: Raspberry Pi, Jetson Nano, PC local, Docker, Python, etc.).

Implementación (Días 2-5):

- Simular la adquisición de la cadena de ADN (puede ser un archivo grande en formato FASTA).
- Dividir la cadena en fragmentos y procesar localmente (por ejemplo, buscar patrones, mutaciones, frecuencia de bases, etc.).

- Enviar solo los resultados relevantes (no toda la secuencia) a un servidor central o la nube.

Documentar la arquitectura y decisiones técnicas.

Pruebas y Optimización (Día 6):

- Medir tiempos de procesamiento, uso de ancho de banda y latencia.
- Validar seguridad en la transmisión de datos.

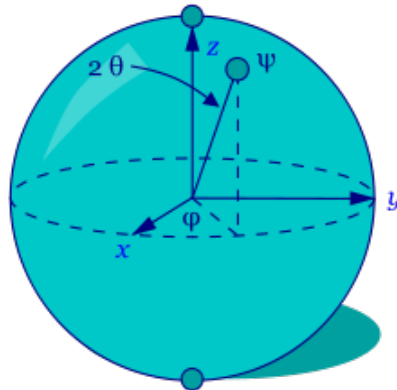
Presentación y Entrega (Día 7):

- Demo funcional del prototipo.
- Informe técnico (10 páginas máx).
- Video de máximo 5 minutos mostrando el funcionamiento y arquitectura.
- Código/documentación en repositorio Git.

Rúbrica de Evaluación (Total: 100 puntos)

Criterio	Excelente (100%)	Bueno (80%)	Regular (60%)	Deficiente (40% o menos)	Puntaje Máx.
1. Diseño de arquitectura EDGE	Arquitectura clara, justificada, y alineada a EDGE Computing real	Arquitectura funcional, con leves omisiones	Arquitectura incompleta o poco justificada	Diseño no aplica EDGE o es confuso	20
2. Implementación funcional	Prototipo corre sin errores, procesamiento local optimizado, resultados válidos	Funciona con algunos fallos menores	Solo procesa una parte de la secuencia o requiere intervención manual frecuente	No ejecuta correctamente o es solo conceptual	25
3. Optimización de transmisión de datos	Solo datos relevantes viajan a la nube, uso eficiente del ancho de banda	Se filtra la mayoría, pero se envían algunos datos innecesarios	Se transmite demasiada información	No hay filtrado; todo va a la nube	15
4. Pruebas, métricas y validación	Se presentan métricas de tiempo, latencia y ancho de banda, y pruebas documentadas	Algunas métricas y pruebas	Métricas básicas o incompletas	Sin métricas ni pruebas	10
5. Seguridad y privacidad de datos	Mecanismos claros de seguridad (cifrado, control de acceso, anonimización, etc.)	Considera parcialmente la seguridad	Considera solo aspectos básicos	No considera seguridad	10
6. Informe y presentación	Informe claro, bien estructurado, y video/demo comprensible	Informe completo pero mejorable en redacción o video	Informe incompleto o poco claro	Presentación muy deficiente o ausente	10
7. Trabajo en equipo y gestión del proyecto	Evidencia clara de colaboración, roles definidos y cumplimiento de cronograma	Colaboración aceptable, algunos roles poco definidos	Participación desigual	Trabajo individual, sin coordinación	10
TOTAL					100

P 4.3 QUANTUM Computing



La [esfera de Bloch](#) es una representación de un [cúbit](#),
<https://blogs.salleurl.edu/es/cloud-vs-fog-vs-edge-computing>

Proyecto: Procesamiento de Cadenas de ADN usando Quantum Computing

Aplicar técnicas de computación cuántica para detectar patrones genéticos en una secuencia de ADN de 4GB, usando circuitos cuánticos simulados o ejecutados en plataformas como IBMQ, Google Quantum AI, Azure Quantum o AWS Braket.

Objetivos Específicos

1. Estudiar los fundamentos de computación cuántica relevantes para búsqueda y optimización.
2. Implementar algoritmos cuánticos como Grover o QAOA para identificar patrones genéticos específicos.
3. Comparar el desempeño entre ejecución en simuladores locales y backends cuánticos reales.
4. Documentar el impacto del ruido cuántico y la escalabilidad en problemas de bioinformática.

Consideraciones Técnicas

- **Entrada:** Secuencia de ADN real o simulada (formato FASTA).
- **Tamaño objetivo:** 4GB (dividir en bloques manejables para ejecución cuántica).

Técnica cuántica sugerida:

- *Grover's Algorithm*: para búsqueda de secuencias específicas (e.g., genes o mutaciones).
- *QAOA o VQE*: para análisis estructural o simulación cuántica de moléculas.

Herramientas y Plataformas Sugeridas

Plataforma	Lenguaje	Acceso	Uso en proyecto
IBM Quantum	Qiskit (Python)	Gratuito (registro)	Simulación y hardware
Google Quantum AI	Cirq (Python)	Acceso limitado	Simulación de circuitos
Azure Quantum	Q# y Python	Gratuito con cuenta educativa	Evaluación con múltiples dispositivos
AWS Braket	Braket SDK (Python)	Gratuito con capa de entrada	Simulación y acceso híbrido

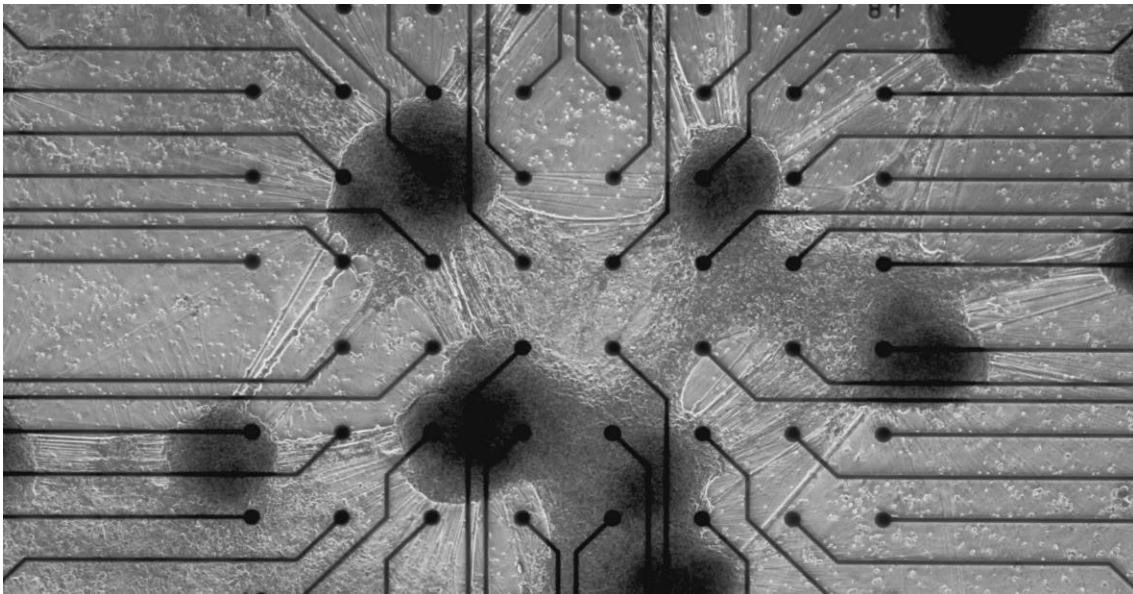
RUBRICA:

Criterio de Evaluación	10	8	6	4	0
1. Fundamentación teórica cuántica	Explica claramente conceptos clave (qubits, puertas, circuitos) y los relaciona con bioinformática.	Incluye definiciones claras y apropiadas, con pocas omisiones.	Presenta conceptos, pero con confusiones o desconexiones.	Información limitada o copiada sin comprensión.	No incluye fundamentos cuánticos.
2. Diseño del circuito cuántico	Diseño lógico, modular, y coherente con el objetivo de análisis de ADN.	Diseño comprensible con leves mejoras posibles.	Diseño parcial o incompleto.	Circuito no adecuado al problema.	No se presenta diseño.
3. Selección e implementación del algoritmo cuántico (Grover, QAOA, etc.)	Selección justificada y ejecución funcional con pruebas.	Algoritmo implementado correctamente con algunos detalles menores.	Algoritmo ejecuta parcialmente o con errores.	Implementación deficiente.	Sin implementación válida.
4. Manejo del dataset de 4GB (fragmentación, preprocesamiento, lectura)	Fragmentación y limpieza óptimas, con integración a la plataforma cuántica.	Manejo adecuado, con pequeños errores.	Manejo parcial de datos.	Limitado o sin aplicación real al tamaño del dataset.	No se usó el dataset correctamente.
5. Ejecución en plataforma cuántica (simulador y/o hardware)	Circuito ejecutado correctamente en simulador y backend cuántico real.	Ejecución en simulador funcional.	Simulación incompleta o limitada.	No logra ejecutar correctamente.	Sin ejecución.
6. Análisis de resultados y comparación clásico/cuántico	Análisis profundo con métricas y visualización clara.	Análisis con resultados correctos y gráficos claros.	Análisis parcial o confuso.	Análisis superficial.	Sin análisis.
7. Integración del equipo y distribución de tareas	Trabajo colaborativo evidente, roles claros.	Buena colaboración, leve desbalance.	Participación irregular.	Distribución deficiente.	No se evidencia trabajo en equipo.
8. Documentación técnica	Completa, con justificaciones, gráficos, referencias y código comentado.	Clara y estructurada, con pequeños vacíos.	Parcial o desorganizada.	Documentación pobre.	No se entrega documentación.
9. Presentación oral o video (defensa del proyecto)	Clara, técnica, dominio del tema y respuestas acertadas.	Buena presentación, con dominio razonable.	Presentación aceptable pero confusa.	Presentación débil.	Sin presentación.

P 4.3 BIOLOGIC Computing



Human Brain Cells Power World's First Commercially Available Biocomputer
https://mikekalil.com/blog/cortical-labs-brain-computer/#google_vignette



Proyecto: “Simulación de procesamiento bio-híbrido de cadenas de ADN de 4GB utilizando redes neuronales biológicas simuladas”

Objetivo del Proyecto:

Diseñar e implementar un prototipo experimental que simule una arquitectura de **computación biológica distribuida**, utilizando modelos de **redes neuronales spiking** para procesar de forma paralela y segmentada una cadena de ADN de 4GB, demostrando su eficiencia y comportamiento frente a un modelo computacional tradicional.

Alcance Técnico:

Componente	Detalle
Entrada	Cadena de ADN de 4GB (formato FASTA o generada sintéticamente)
Procesamiento Bio	Simulación de redes neuronales tipo spiking (SNN) con BindsNET o NEST
Comparador tradicional	Algoritmo base secuencial o procesamiento paralelo en Python
Arquitectura propuesta	Dividir tareas en nodos virtuales simulando comportamiento sináptico
Salida	Reporte de eficiencia en tiempo, precisión, consumo de recursos

Tecnologías sugeridas:

- **Lenguaje:** Python 3.x
- **Librerías:** Biopython, BindsNET, NEST, NumPy, Pandas
- **Plataforma:** Docker (simulación de nodos), Jupyter, entorno Linux
- **Procesamiento distribuido** (opcional): Ray o MPI para escalar análisis

Alcance:

Día

Actividades principales

- 1 Revisión de concepto SNN y carga de la cadena de ADN (sintética o FASTA real)
- 2 Implementación del preprocesamiento del ADN y segmentación en fragmentos
- 3 Simulación de red spiking en BindsNET con una sola capa de neuronas
- 4 Comparación con un algoritmo clásico de búsqueda/matching en Python
- 5 Análisis de resultados, visualización y presentación de informe

Rúbrica de Evaluación (50 puntos totales)

Criterio	Descripción	Puntaje
1. Diseño del prototipo bio-híbrido	Estructura lógica, justificación del modelo neuronal simulado, arquitectura modular	/10
2. Desarrollo y funcionalidad del sistema	Correcta simulación de redes SNN y comparación con sistema tradicional	/10
3. Procesamiento de cadenas ADN 4GB	Eficiencia del algoritmo, correcta partición de datos, uso de herramientas bioinformáticas	/10
4. Innovación y aplicación de tendencias	Originalidad en el uso de computación biológica simulada, integración con conceptos actuales	/10
5. Informe técnico y defensa del proyecto	Claridad, documentación, presentación de resultados, análisis crítico y defensa oral	/10

OPEN QUAKE [RETO de EXONERACION]

OpenQuake es un software de código abierto para evaluación de AMENAZA y riesgo sísmico. URL: <https://www.training.openquake.org/>



Se lo tomará como una “caja negra” que produce resultados en un archivo y que requiere poder computacional para un GRID especificado en Km2.

A continuación, el correo del Ing. Carlos Celi autor principal de dicho software que contiene un .RAR para su implementación y ejecución, ubicado en la carpeta compartida REFERENCIAS BIBLIOGRAFICAS para Computación PDN.

Se sugiere leer detenidamente el correo siguiente y los comentarios indicados en el archivo.rar

Existen dos ejemplos a ejecutarse:

- a. Ejemplo corto [GRID 50]
- b. Ejemplo largo [GRID 5]

Estimado Ingeniero Córdor,

Como conversamos adjunto, las carpetas con sus archivos .ini respectivos para correr en el software OpenQuake. es importante recordarles a los estudiantes que pueden descargar el software desde el [GitHub](#) y pueden encontrar la documentación respectiva en la página [web](#) donde pueden encontrar todos los pasos para la instalación.

También es importante recordarles a los estudiantes que para ejecutar el archivo '.ini' deben abrir tanto el "OpenQuake Engine (console)" y "OpenQuake Engine (webui)" y las carpetas que adjunto deben ser colocadas en donde se instaló el programa, específicamente dentro de la carpeta "C:\ \OpenQuake Engine\demos\hazard". dentro de las carpetas puse un archivo '.txt' donde ya está el comando que deben ingresar en "OpenQuake Engine (console)" para iniciar la ejecución de los cálculos.

Como se habló en su clase, también puede realizar el ejercicio con base a un programa más simple, el cual lo pueden encontrar en mi [GitHub](#), al cual también pueden acceder desde la mi página [web](#) dirigiéndose al apartado respectivo.

Saludos cordiales.

MSc. Ing. Carlos Celi

Profesor Titular

Facultad de Ingeniería/ Carrera: Ingeniería Civil

Pontificia Universidad Católica del Ecuador

Ext: 1978

<https://caceli.net>

https://www.researchgate.net/profile/Carlos_Celi

<https://scholar.google.com/citations?user=yR4Gz7kAAAAJ&hl=es>

<https://www.puce.edu.ec/>

2. Defender la implementación de

- 2.1 OpenQuake optimizado en Cloud Computing
- 2.2 Una implementación en FOG Computing
- 2.3 Una implementación en EDGE Computing

3. Exponer una investigación sobre:

- 3.1 QUANTUM Computing - limitaciones físicas
- 3.2 QUANTUM Computing - limitaciones matemáticas
- 3.3 QUANTUM Computing vs BINARY Computing

4. En caso de tener la IMPLEMENTACION FINAL, se debe proceder con la realización de los 3 proyectos y dar los exámenes finales.